

Bericht RLE

Bericht zur Minichallenge im Modul Reinforcement Learning

Herbstsemester 25

Projekt von: Adrian Meier

Zuständiger Fachexperte: Yanick Schraner

Ort, Datum: Brugg, 07.01.2026

Inhalt

Einleitung.....	3
Baseline	3
Initialer Ansatz.....	4
Implementierung.....	4
Experimentelles Setup.....	6
Evaluationsrezept	7
Erweiterungen	8
Double DQN.....	8
Hypothese	9
Implementierung.....	9
Evaluation	9
Analyse	10
Dueling DQN.....	10
Hypothese	10
Implementierung.....	10
Evaluation	11
Analyse	12
PER Replay Buffer	12
Hypothese	12
Implementierung.....	12
Evaluation	13
Analyse	14
Noisy Net	14
Hypothese	14
Implementierung.....	14
Evaluation	15
Analyse	15
Finaler Vergleich	16
Fazit	17
Ausblick	17

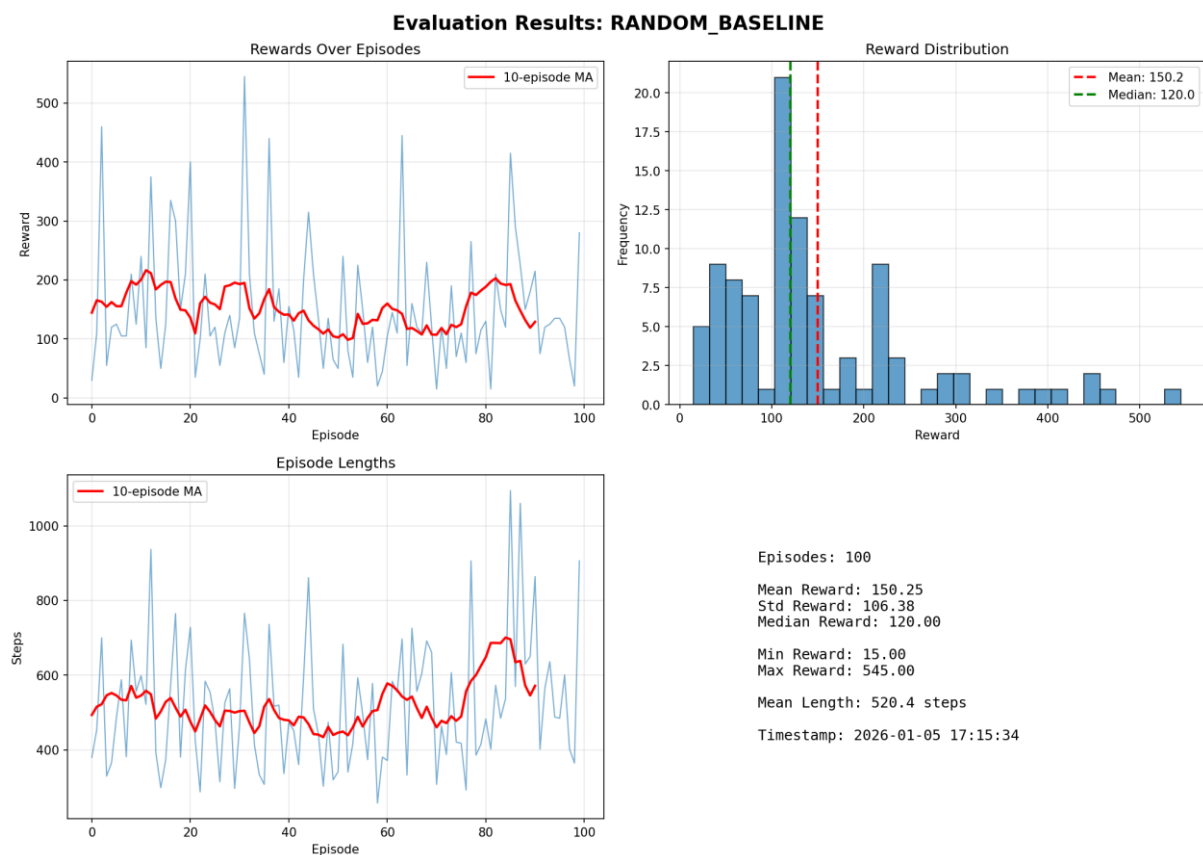
Einleitung

Dies ist der Bericht zur Minichallenge im Reinforcement Learning Modul. Das Ziel ist es, einen DQN-Agenten zu bauen und dann vier Erweiterungen zu testen und zu sehen, wie sie sich auswirken. Der komplette Code befindet sich in einem GitHub Repository und ist verfügbar unter <https://github.com/adrianmeier2002/RLE> öffentlich zugänglich. Um den Code auszuführen, verweise ich auf die Datei README.md im Repository, die eine kurze Anleitung zum Ausführen der Dateien enthält.

Baseline

Als Baseline wurde ein Random Agent gewählt. Dieser führt zufällige Aktionen aus und hat kein Gedächtnis. Dies ist ein sehr einfacher Ansatz, aber es zeigt sich auch, dass im Spiel Space Invaders mit rein zufälligen Aktionen eine gewisse Punktzahl erreicht werden kann.

Der Aufbau des Agenten ist relativ einfach, der Agent nimmt sich immer eine zufällige Aktion aus dem Aktionsraum und führt diese aus. Es gibt keine Aufzeichnung darüber, welche Aktionen er ausgeführt hat, was gut war und was nicht und auch kein sonstiges Feedback für den Agenten. Da alles nach dem Zufallsprinzip abläuft, muss der Agent auch nicht wirklich trainiert werden. Es werden lediglich mehrere Durchläufe durchgeführt, um die Streuung der Ergebnisse sauber dokumentieren zu können. Die Ergebnisse geben Auskunft darüber, was mindestens erreicht werden kann, ohne etwas gelernt zu haben.



Als ich die Ergebnisse zum ersten Mal gesehen habe, war ich etwas überrascht, wie gut das Spiel teilweise mit zufälligen Aktionen gespielt werden kann. Die Analyse der Grafiken zeigt jedoch auch, dass die Ergebnisse enorm variieren und auch die Anzahl der ausgeführten Schritte sehr unterschiedlich ist. Es zeigt sich, dass mit zufälligen Aktionen das Spiel prinzipiell spielbar ist, die Belohnungen aber bis auf wenige Ausnahmen zwischen 0 und 220 liegen.

Initialer Ansatz

Als initialen Ansatz habe ich ein DQN (Deep Q-Netzwerk) implementiert. Als Ausgangspunkt habe ich das GitHub Repository von Yannick Schraner ([YannickSchraner/rle-mini-challenge](#)). Die komplette Implementierung habe ich dann aber selbst gemacht. Der Grund dafür war zum einen, dass ich so den Code besser verstanden habe, was wie zusammenarbeitet und zum anderen, weil ich selbst einmal eine solche Implementierung schreiben wollte. Das hat sich später als praktisch erwiesen, da ich die Erweiterungen schnell einbauen konnte, da ich den Code des ursprünglichen Ansatzes besser verstanden habe.

Ich habe mich für DQN entschieden, weil ich einen grossen Zustandsraum habe (Pixel können sehr unterschiedlich sein), aber andererseits nur wenige diskrete Aktionen (links, rechts, schiessen). Ausserdem ist das Konzept von DQN für mich intuitiver und verständlicher als Policy Gradient.

Implementierung

Der initiale Ansatz besteht im Wesentlichen aus drei Komponenten. In **dqn_initial.py** wird das Neuronale Netz mit drei Schichten und einer Normalisierung der Pixelwerte implementiert, der Agent selbst erstellt und das Training durchgeführt.

Die gewählte Netzwerkarchitektur ist speziell an die Eigenschaften dieser Umgebung angepasst und basiert auf der bewährten Architektur der ursprünglichen DeepMind-DQN Arbeit.

Das Eingangssignal besteht aus Bilddaten mit der Form (Channels, Height, Width), typischerweise aus mehreren gestapelten Graustufenbildern. Da der Zustandsraum dadurch sehr hochdimensional ist, wird ein Convolutional Neural Network (CNN) verwendet, um die relevanten visuellen Merkmale effizient zu extrahieren.

Convolutional Layer

Das Netz beginnt mit drei aufeinander folgenden Faltungsebenen:

1. Der erste Convolutional Layer mit einer Kernelgrösse von 8×8 und einem Stride von 4 reduziert die räumliche Auflösung stark und extrahiert grobe, bewegungsrelevante Strukturen wie Gegnerformationen oder Projektilbewegungen.
2. Die zweite Schicht (4×4 Kernel, Stride 2) verfeinert diese Merkmale und erkennt komplexere Muster.
3. Die dritte Schicht (3×3 Kernel, Stride 1) dient der präzisen Merkmalsextraktion und der Stabilisierung der Repräsentation.

Durch diese Architektur wird eine schrittweise Abstraktion der visuellen Information erreicht, die besonders für Spiele mit klaren Objektstrukturen wie Space Invaders geeignet ist.

Fully Connected Layer

Nach den Convolutional Layers werden die extrahierten Merkmale flach gemacht und an ein Fully-Connected-Network weitergegeben:

- Eine Dense-Schicht mit 512 Neuronen ermöglicht es dem Modell, komplexe nichtlineare Zusammenhänge zwischen dem Spielzustand und den möglichen Aktionen zu lernen.
- Die finale Ausgabeschicht liefert für jede mögliche Aktion einen Q-Wert, der den erwarteten zukünftigen Reward dieser Aktion repräsentiert.

Diese Struktur entspricht direkt der Q-Funktion $Q(s, a)$, die im DQN-Ansatz approximiert wird. Die Pixelwerte werden zudem noch normalisiert. Dadurch sollen die Eingaben in einem numerisch stabilen Bereich liegen.

Im DQNAgent wird dann der Agent selbst erstellt. Hier wird die Netzwerkarchitektur definiert und die Epsilon Greedy Policy eingestellt, um am Anfang viel zu explorieren und gegen Ende mehr auf bewährte Aktionen zu setzen. Für die Aktualisierung wird die Bellman-Gleichung mit einem Huber-Loss berechnet. Ich habe mich gegen einen Mean-Squared-Error entschieden, da der Huber-Loss etwas robuster ist und grosse Fehler nicht exponentiell bestraft und somit die Übergänge nicht zu extrem werden lässt. Zusätzlich habe ich noch ein Clipping implementiert, um eine bessere Stabilität im Training zu haben.

Schliesslich wird das Modell trainiert. Mit Hilfe von Schleifen sammelt der Agent Erfahrungen und speichert und aktualisiert seine Parameter. Während des Trainings werden die Metriken in einem Tensorboard gespeichert und können dargestellt werden.

Neben diesem Hauptprogramm gibt es noch zwei weitere wichtige Programme. Der **replay_buffer.py** und der **env_utils.py**. Im Replay Buffer werden Tupel mit state, action, reward, next_state und done gespeichert und dann zufällig gesampelt.

Für die Erstellung eines Environments benötige ich die Datei **env_utils.py**, die sich um das Sammeln der Bewegungsinformationen kümmert. Ausserdem habe ich dort einen Framestack, also die Informationen, die gestapelt werden, um die Bewegungen erfassen zu können. In meiner Umgebung habe ich einen Outputshape von (4, 84, 84), was dem Framestack und der Anzahl der Pixel entspricht. Weiter habe ich dort auch noch einen Frameskip implementiert. Dieser sorgt dafür, dass gewisse Frames übersprungen werden. Das macht das Training etwas schneller, dafür auch etwas ungenauer.

Experimentelles Setup

Für das Training aller fünf DQN-Varianten wurde ein einheitlicher Versuchsaufbau verwendet, um eine faire Vergleichbarkeit der Ergebnisse zu gewährleisten. Die zentralen Hyperparameter sind in der Datei **train_all.py** ab Zeile 202 sowie teilweise direkt in den jeweiligen DQN-Implementierungen definiert. Die Tabelle gibt einen Überblick über die verwendeten Parameter.

Hyperparameter	Wert
num_steps	5'000'000
batch_size	32
target_update_freq	10'000
learning_starts	50'000
train_freq	4
video_every	100
eval_freq	100000
eval_episodes	10
Learning Rate	1e-4
Replay Buffer Capacity	100'000
epsilon_start	1.0
epsilon_final	0.01
gamma	0.99
epsilon_decay	1'000'000
Frameskip	4

Die Wahl dieser Hyperparameter orientiert sich an etablierten Konfigurationen aus der DQN-Literatur, insbesondere an erfolgreichen Anwendungen in Atari-Umgebungen, und hat sich in der Praxis als stabil und effizient erwiesen.

Die Wahl für die Anzahl der Schritte kam daraus, dass dies ein Training in einer angemessenen Zeit darstellte und eine für mich akzeptable Anzahl an Episoden gespielt wurden. Zudem habe ich vor dem definitiven Training der Agenten einen Testlauf mit einer früheren Version des initialen Ansatzes über 20 Millionen Schritte gemacht. Auch dort hat sich gezeigt, dass der Lernfortschritt nach 5 Millionen Schritten etwas stagniert ist.

Die Lernrate von 1×10^{-4} stellt einen Kompromiss zwischen Lernstabilität und Konvergenzgeschwindigkeit dar. Sie ist klein genug, um starke Oszillationen oder divergierendes Verhalten zu vermeiden, erlaubt aber gleichzeitig eine kontinuierliche Anpassung der Netzgewichte.

Die Batchgrösse von 32 wurde gewählt, da sie eine ausreichend stabile Gradientenschätzung ermöglicht, ohne die Varianz der Aktualisierungen unnötig zu erhöhen. Gleichzeitig bleibt der Speicher- und Rechenaufwand moderat.

Der Replay Buffer hat eine Kapazität von 100'000 Transitionen, wodurch eine grosse Anzahl von Zustandsübergängen gespeichert und zufällig abgetastet werden kann. Dies reduziert Korrelationen zwischen aufeinanderfolgenden Samples und trägt wesentlich zur Stabilisierung des Lernprozesses bei.

Der Discountfaktor $\gamma = 0.99$ sorgt dafür, dass zukünftige Belohnungen stark berücksichtigt werden, was insbesondere in Space Invaders wichtig ist, da erfolgreiche Strategien auf langfristigem Überleben und wiederholtem Abschiessen von Gegnern basieren.

Zur Steuerung der Exploration wird ein ϵ -greedy-Verfahren eingesetzt. Der Epsilon-Wert wird von 1.0 auf 0.01 über den Zeitraum des Trainings reduziert. Dadurch wird zu Beginn des Trainings eine intensive Exploration des Zustandsraums ermöglicht, während das Verhalten gegen Ende zunehmend deterministisch und exploitativ wird.

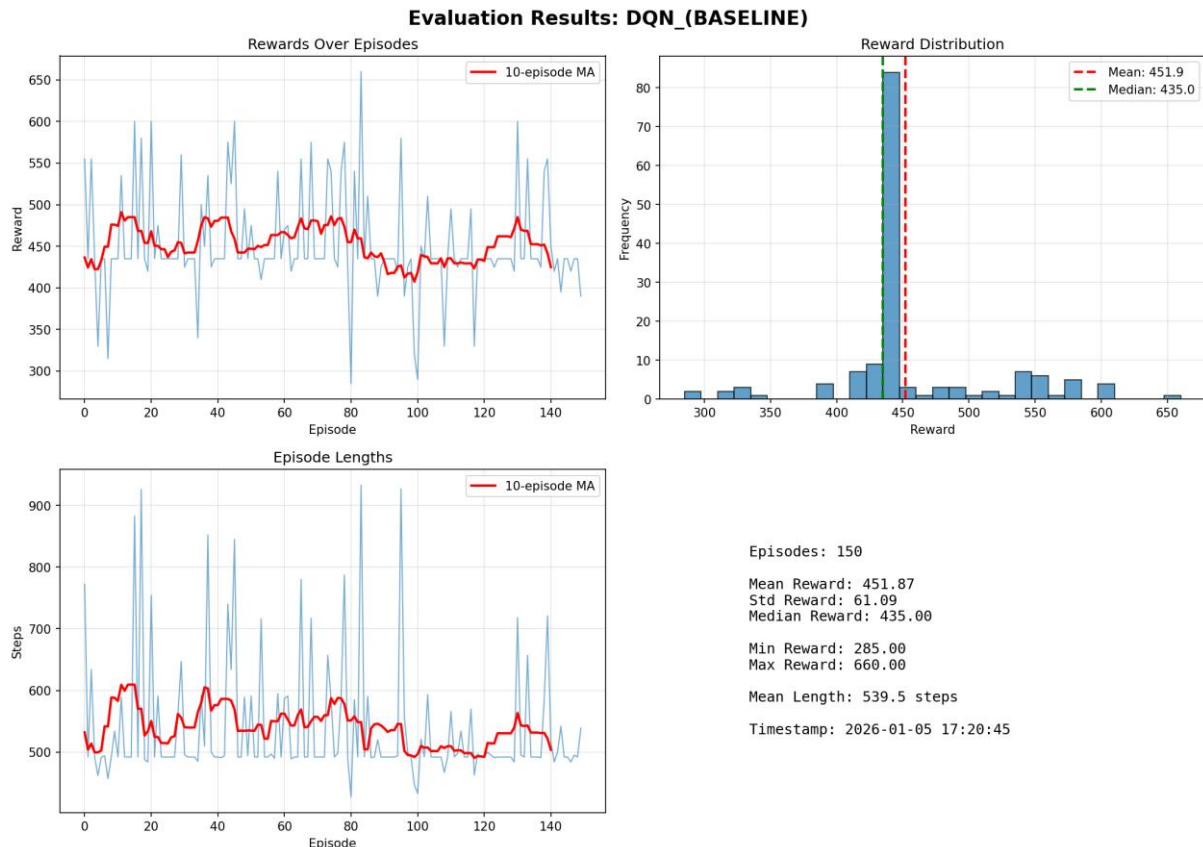
Insgesamt stellt dieses Setup eine bewährte und gut balancierte Konfiguration dar, die sowohl eine stabile Lernphase als auch eine aussagekräftige Evaluation der verschiedenen DQN-Varianten erlaubt.

Evaluationsrezept

Die Auswertung habe ich mit Hilfe der **eval_utils.py** durchgeführt. Es werden jeweils 150 Episoden bzw. 100 Episoden für den Random Agent gespielt. Diese Anzahl sollte ein klares Bild davon geben, wie gut der Agent spielt.

Bei der Auswertung hatte ich das Problem, dass wenn ich Epsilon auf Null setzte, die Ergebnisse für jede Episode gleich waren. Das lag daran, dass die Umgebung immer am gleichen Ort gestartet wurde. Ich habe versucht, das mit verschiedenen Seeds zu beheben, aber das hat nicht funktioniert. Schliesslich habe ich mit einem kleinen Epsilon von 0,01 gearbeitet. Dadurch erhalte ich etwas andere Ergebnisse und kann etwas darüber aussagen, wie gut oder schlecht bestimmte zufällige Aktionen das Modell beeinflussen könnten.

Am Ende jeder Bewertung wird ein Diagramm für jedes Modell und eine Zusammenfassung für alle Modelle erstellt.



Dies ist z.B. die Auswertung der Initialen DQN. Man sieht, dass es einen klaren Median bei 435 gibt, was das Ergebnis zu sein scheint, wenn nie zufällig gehandelt wurde. Es ist aber auch interessant zu sehen, dass durch zufälliges Handeln des Agenten nicht nur schlechtere, sondern auch einige bessere oder sogar viel bessere Ergebnisse erzielt werden konnten. Auch die Anzahl der Schritte zeigt, dass im Normalfall ca. 500 Schritte gemacht werden, an einigen Stellen aber auch deutlich mehr. Teilweise wurden bis zu 900 Schritte in einer Episode gemacht. Es scheint noch viel Luft nach oben im ursprünglichen DQN zu geben, vor allem wenn man sieht, dass einige zufällige Aktionen die Belohnung noch einmal deutlich erhöht haben.

Erweiterungen

Für die Erweiterung des DQN habe ich vier verschiedene Neuerungen implementiert. Nachfolgend wird jede Implementierung kurz erklärt, meine Hypothese, was passiert, wie ich es implementiert habe, wie die Evaluation aussah und ein kurzes Fazit.

Double DQN

Im ursprünglichen DQN-Ansatz besteht das Problem, dass dasselbe Zielnetz die beste Aktion auswählt und diese direkt bewertet. Dies führt zu einer Überschätzung der Q-Werte. Der Agent wählt also die Aktionen aus, die vom Targetnet zufällig hoch bewertet wurden und nicht, weil sie wirklich gut sind.

Double DQN erzeugt zwei Netzwerke. Ein Q-Netz, das die beste Aktion auswählt und ein Targetnet, das die beste Aktion bewertet. Dadurch wird die Aktion oft nicht mehr überschätzt. Ziel ist es, die Konvergenz zu verbessern und das Training zu stabilisieren.

Hypothese

Das Training sollte ruhiger laufen. Die Updates sollten sich aufgrund von weniger Überschätzung in der Aktionsauswahl weniger stark auswirken. Ich gehe davon aus, dass die Lernkurve weniger sprunghaft ist und vielleicht etwas besser konvergiert.

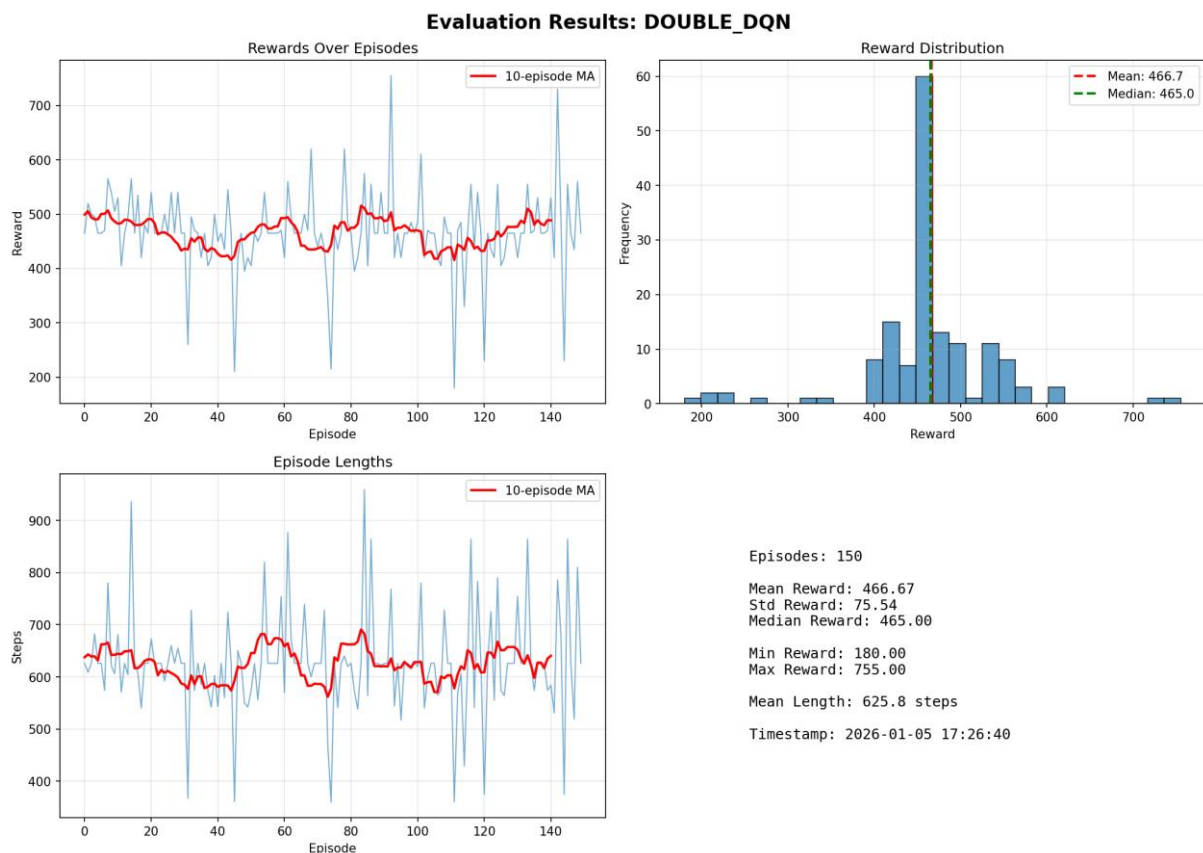
Implementierung

Die Implementierung erfolgte in der Datei **dqn_double.py**. Im Wesentlichen wurde nur ein kleiner Teil in der Berechnung und Auswertung der Q-Werte durchgeführt. Die nächste Aktion wird aus dem `q_net` genommen, aber der Q-Wert selbst wird dann aus dem `target_net` genommen.

```
# Q(s,a)
q_values = self.q_net(states).gather(1, actions)

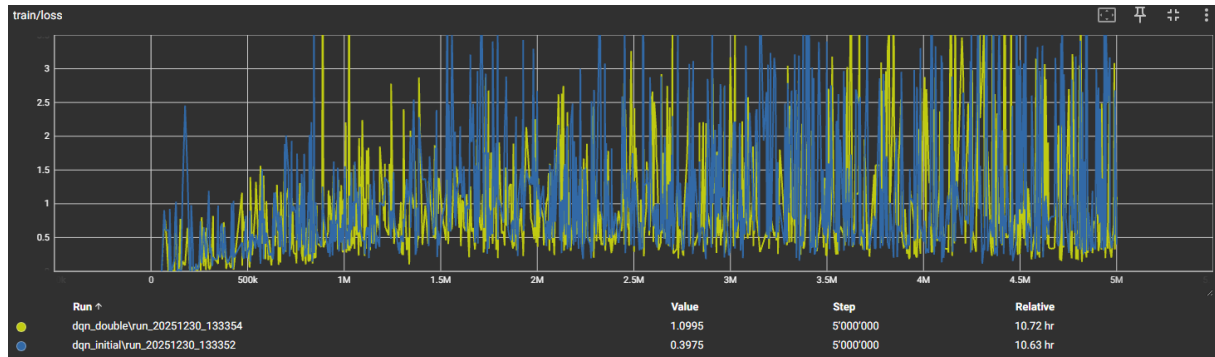
with torch.no_grad():
    # Double DQN: action selection from q_net, evaluation from target_net
    next_actions = torch.argmax(self.q_net(next_states), dim=1, keepdim=True)
    next_q_values = self.target_net(next_states).gather(1, next_actions)
    target = rewards + self.gamma * next_q_values * (1 - done)
```

Evaluation



In der Evaluation des Double DQN sehen wir sehr ähnliche Werte und Zahlen wie beim Initialen Ansatz. Das Model ist leicht besser gerade im Mittelwert und im Median. Es führt zudem im Schnitt etwa 100 Schritte mehr aus als der klassische Ansatz.

Für meine Hypothese ist aber vor allem auch das Training interessant. In meinen Augen sollte es nun einen klaren Unterschied zwischen dem initialen und dem double Ansatz geben, mit weniger Spikes. Jedoch ist im Tensor Board davon wenig zu erkennen. Auch die anderen Grafiken im Tensor Board lassen keine klaren Unterschiede erkennen.



Analyse

Im Vergleich zum DQN sollte dieser Agent dafür sorgen, dass die Q-Werte weniger überschätzt werden und der Antrieb etwas stabiler ist. In meinem Fall hat sich kein signifikanter Unterschied gezeigt. Nur der Reward hat sich ein wenig erhöht. Man kann aber auch sagen, dass durch die Double DQN sicher keine Verschlechterung eingetreten ist.

Dueling DQN

Das Ziel von Dueling DQN ist die Trennung von Aktion und Zustand. Dueling DQN lernt, wie gut ein Zustand grundsätzlich ist und wie gut eine Aktion in einem Zustand ist. Dazu wird das Convolutional Layer aufgeteilt. Und der Q-Wert wird am Ende rekonstruiert.

Hypothese

Das Modell lernt auch die Zustände unabhängig von der ausgeführten Aktion. Dadurch können die Q-Werte etwas stabiler werden.

Implementierung

Das Dueling Network hat nun zwei getrennte Netzwerke für State Values (value) und für Action Values (advantage).

```

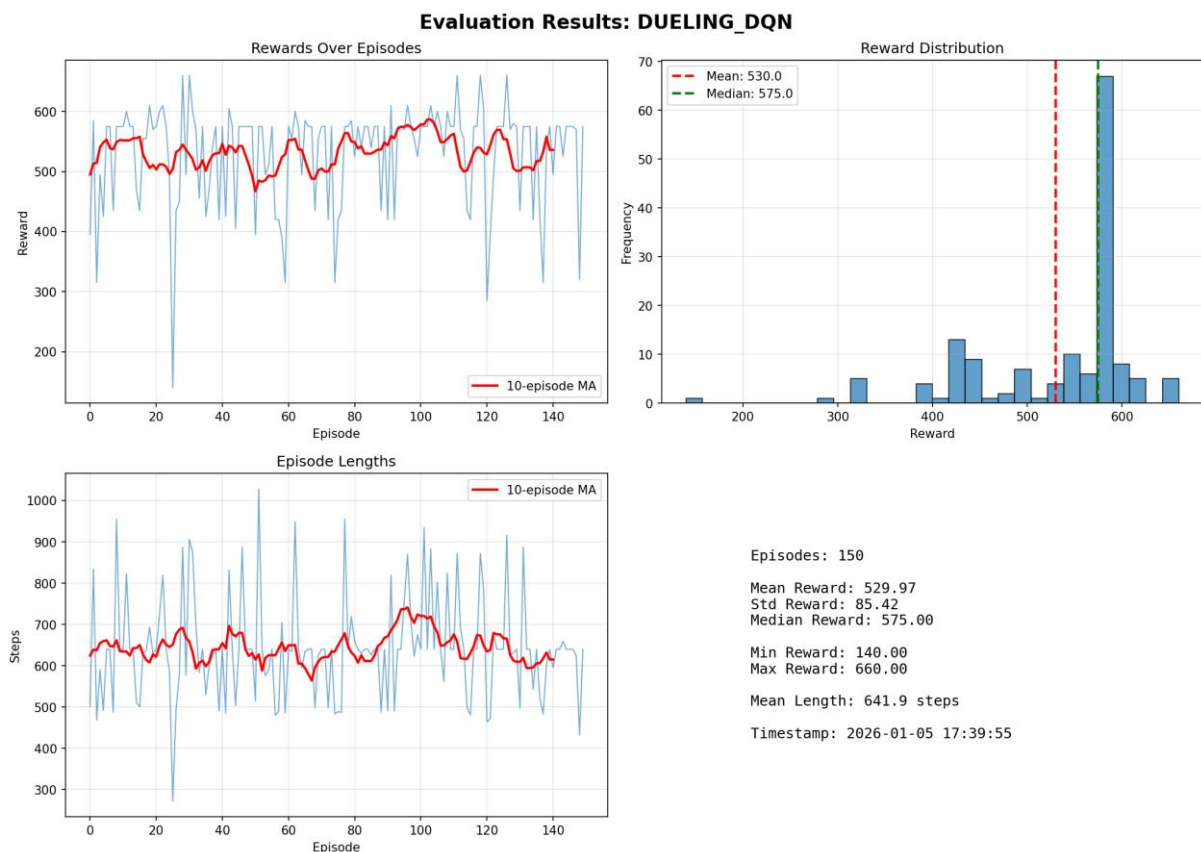
# Value stream
self.value = nn.Sequential(
    nn.Linear(conv_out_size, 512),
    nn.ReLU(),
    nn.Linear(512, 1)
)

# Advantage stream
self.advantage = nn.Sequential(
    nn.Linear(conv_out_size, 512),
    nn.ReLU(),
    nn.Linear(512, num_actions)
)

```

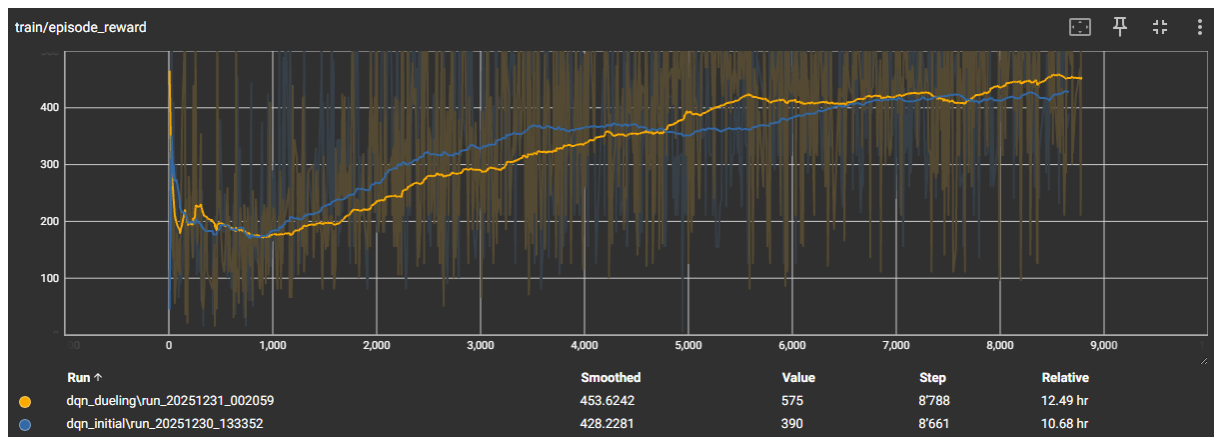
Zusätzlich werden im Agenten ein `q_net` und ein `target_net` mit den neuen Netzen initialisiert.

Evaluation



Der Dueling Agent ist deutlich besser als der initiale Ansatz. Er hat einen deutlich höheren Reward. Er hat jedoch auch einen grösseren Ausreisser in der Auswertung. Die Episoden des Agenten sind auch länger als die des DQN.

Im Tensor Board sind keine gravierenden Unterschiede zu erkennen. Jedoch ist in den Rewards der Episoden zu sehen, dass der Reward anfangs etwas niedriger ist, dafür stetiger ansteigt.



Analyse

Das Double DQN kann den Reward erhöhen. Es dauert dafür etwas länger und hat zumindest in meiner Evaluation einen grösseren Einfluss durch zufällige Aktionen zwischendurch. Spannend ist zudem, dass es in einigen Videos des Double DQN Agenten so aussieht, als ob er sich hinter dem Schutz verstecken würde. Dies würde darauf hindeuten, dass er gelernt hat, dass dies ein guter Zustand für ihn ist, da er dort nicht getroffen werden kann.

PER Replay Buffer

Der Prioritized Experience Replay Buffer ist eine Erweiterung der normalen Replay Buffers. Er sorgt dafür, dass die wichtigen Übergänge häufiger trainiert werden als die unwichtigen. Dies geschieht durch eine Berechnung des TD-Fehlers und eine anschließende Priorisierung.

Hypothese

Der Agent soll schneller lernen und besser und gezielter mit unerwarteten Rewards, seltenen Ereignissen und falschen Q-Schätzungen umgehen können. Dadurch können Ereignisse wie der Tod im Spiel besser erfasst werden. Kritische Situationen können so besser verstanden werden.

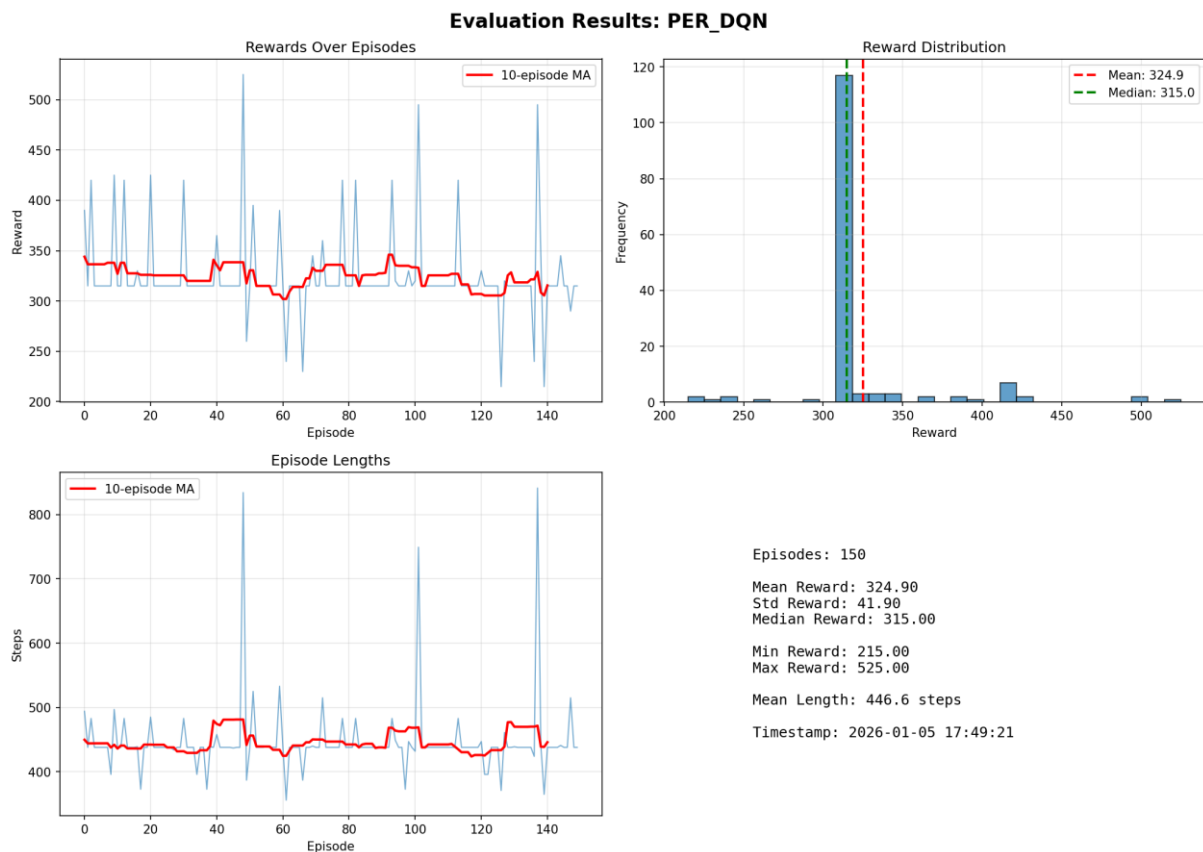
Implementierung

Für die Implementierung wurde ein neuer Buffer erstellt. Dieser wird dann von einem Agenten genutzt. Die Hauptänderung besteht darin, dass Prioritäten berechnet und beim Sampling berücksichtigt werden.

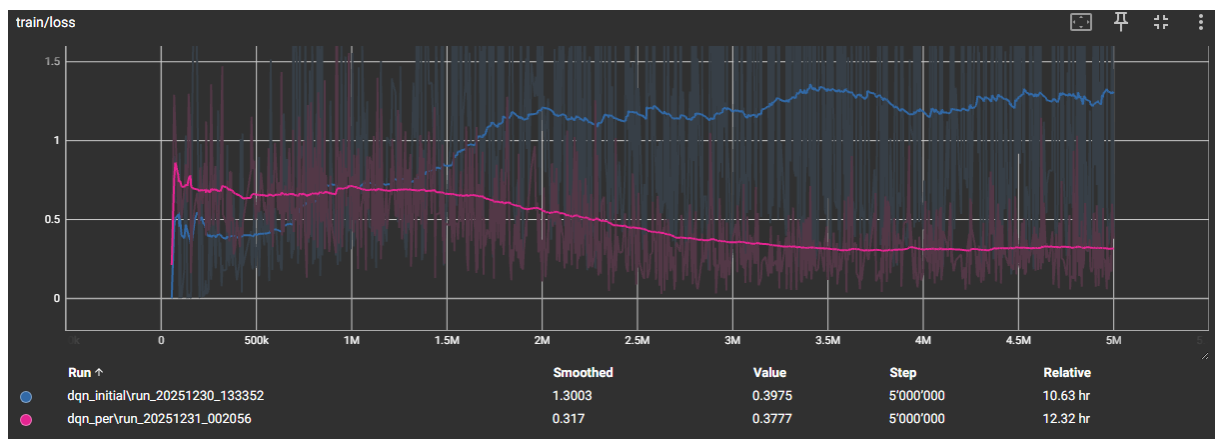
```
def update_priorities(self, idx, td_errors):
    """Update priorities using TD-error."""
    if isinstance(td_errors, torch.Tensor):
        td_errors = td_errors.detach().cpu().numpy()
    td_errors = td_errors.flatten()

    self.priorities[idx] = np.abs(td_errors) + self.epsilon
```

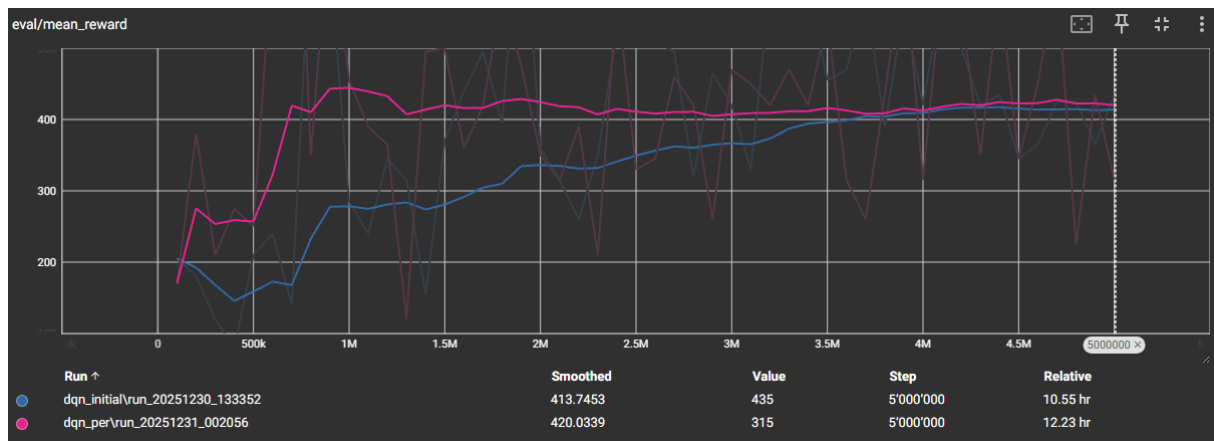
Evaluation



Der PER-Agent performt deutlich schlechter als die anderen Agenten. Auch die Anzahl der gemachten Schritte ist nicht sonderlich hoch. Im Tensor Board ist gut zu sehen, dass zu beginn viel angepasst wird und danach der Loss abflacht. Dies könnte im Anschluss problematisch gewesen sein, da der Agent auf bereits bekannten Patterns Overfitten könnte.



Es ist gut zu sehen, dass der Reward während des Trainings sehr schnell stagniert. Auch dies deutet auf das Overfiting hin und, dass er nur noch auf Ähnlichem trainiert.



Analyse

Der PER Buffer hat nicht mein gewünschtes Ziel erreicht. Ich habe gesehen, dass gerade der Loss sehr schnell abnimmt und er dadurch keine grossen Fortschritte mehr machen kann. Meine Hoffnungen, dass er seinen Tod vielleicht etwas hinauszögern kann, haben sich auch nicht bestätigt, im Gegenteil, er hat eine deutlich kürzere Schrittlänge als die anderen.

Noisy Net

Noisy Networks (Noisy Nets) sind eine Explorationsmethode des Reinforcement Learning, bei der die Zufälligkeit direkt in die Gewichtung des Netzes einfließt. Anstatt zufällige Aktionen auszuwählen, wie es bei der Epsilon-Greedy-Methode der Fall ist, erzeugen Noisy Nets zufällige, aber strukturierte Variationen in den Q-Werten. Dadurch wird die Exploration zustandsabhängig und konsistenter über mehrere Zeitschritte. Die Stärke des Rauschens kann gelernt werden, so dass das Netz selbst entscheidet, wie viel Exploration erforderlich ist. Ein wesentlicher Vorteil von Noisy Net besteht darin, dass keine manuelle Steuerung eines ϵ -Decay-Schemas erforderlich ist. Im Vergleich zu ϵ -greedy werden weniger unnötige Zufallsaktionen ausgeführt, was die Effizienz der Abtastung erhöht.

Hypothese

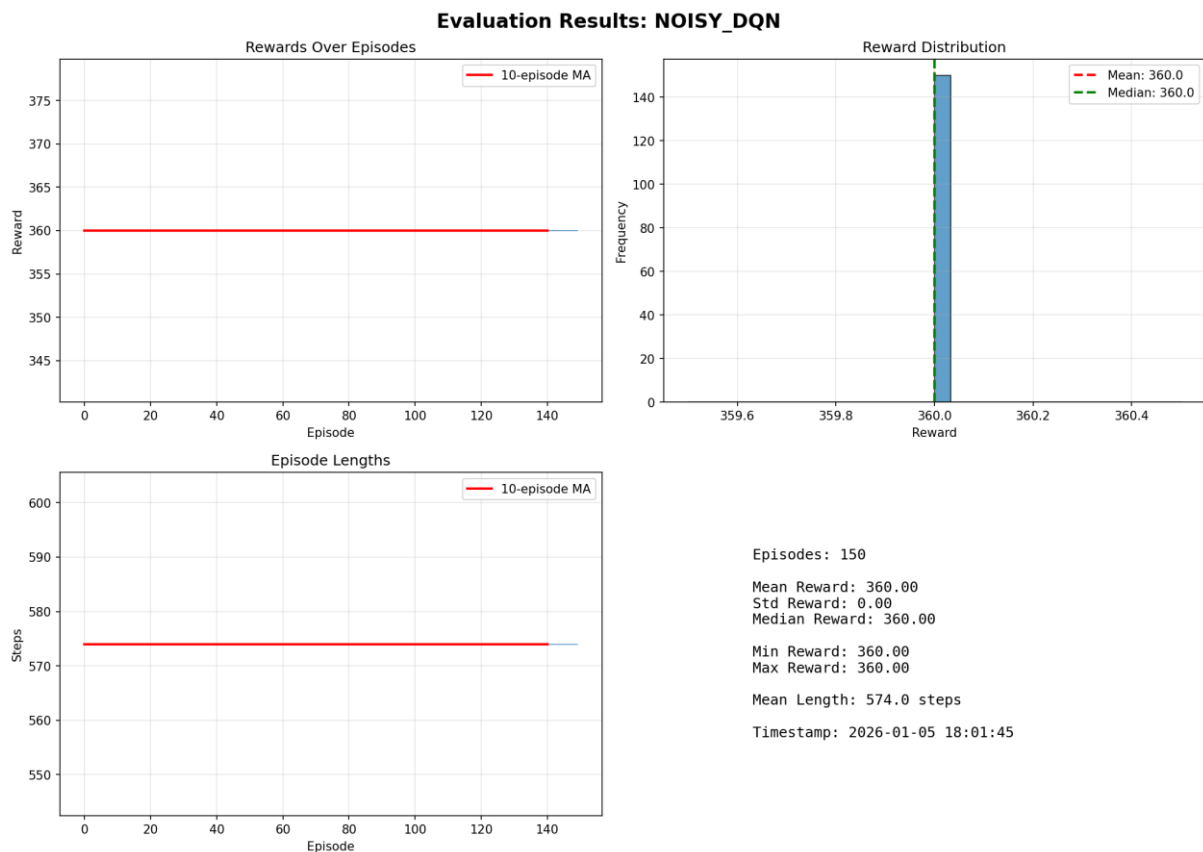
Die Verwendung von Noisy Networks anstelle der klassischen ϵ -greedy Exploration im ursprünglichen DQN-Ansatz führt zu einer effizienteren und zielgerichteteren Exploration, wodurch der Agent schneller lernt und eine höhere durchschnittliche Spielleistung erreicht.

Implementierung

Für die Implementierung habe ich den Agenten in **dqn_noisy** erstellt. Die wichtigsten Änderungen sind die Layer, die das Rauschen hinzufügen. Alles, was mit Epsilon zu tun hatte, wurde ersetzt.

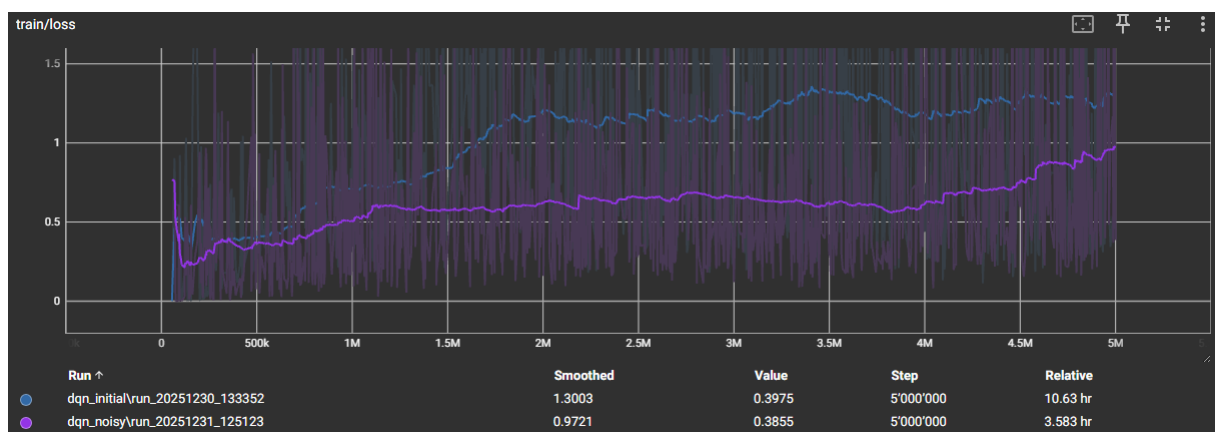
```
def forward(self, x):
    if self.training:
        weight = self.weight_mu + self.weight_sigma * self.weight_epsilon
        bias = self.bias_mu + self.bias_sigma * self.bias_epsilon
    else:
        weight = self.weight_mu
        bias = self.bias_mu
    return F.linear(x, weight, bias)
```

Evaluation



Da es im Noisy Network kein Epsilon gibt, das zufällige Aktionen ausführen kann, bleibt der Reward immer gleich.

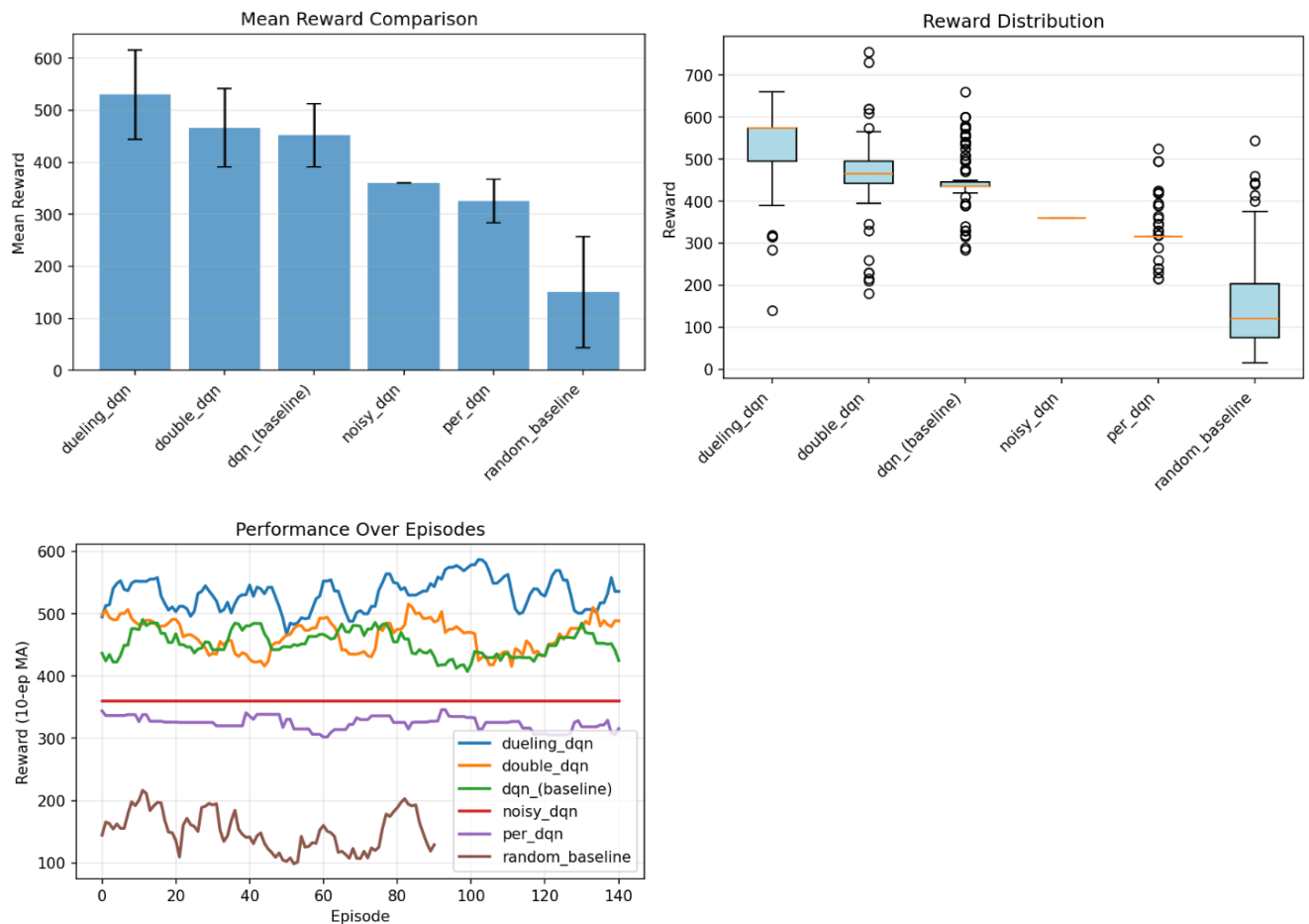
Es ist schnell ersichtlich, dass auch dieser Ansatz nicht besonders gut ist und deutlich unter dem ursprünglichen Ansatz liegt. Es ist auch zu erkennen, dass der Verlust geringer ist als beim ursprünglichen Ansatz und nach etwa einer Million Schritte zu stagnieren beginnt.



Analyse

Der Ansatz ist sehr interessant, da er das Problem anders angeht als alle anderen Methoden. Es ist aber auch klar, dass er nach einer gewissen Zeit an seine Grenzen stösst und es dann schwierig ist, weiter zu lernen.

Finaler Vergleich



Es ist sicherlich interessant zu sehen, dass unabhängig vom gewählten Agenten und der gewählten Verbesserung die Performance immer besser ist als beim Random Agenten. Es ist auch ersichtlich, dass einige der Agenten etwas mehr Streuung in den Ergebnissen haben als andere.

Es ist aber auch ersichtlich, dass die experimentelleren Ansätze des Noisy Networks und des PER Buffers schlechter abschneiden als der Initiale Ansatz.

Es wird auch deutlich, dass zumindest in den von mir durchgeführten Experimenten das Dueling-DQN eindeutig am besten abschneidet.

Fazit

Ich habe während dieser Challenge sehr viel über DQN und Reinforcement Learning gelernt. Man erkennt, wie kleine Änderungen, wie z.B. eine andere Betrachtung von Zustand und Aktion, einen grossen Einfluss auf den Agenten haben können.

Ausserdem wird einem bewusst, wie viel Rechenleistung und Zeit ein solches Programm benötigt. Es hat mir gezeigt, dass es für bestimmte Probleme sehr nützlich sein kann, ein DQN zu erstellen, aber man muss sich auch darüber im Klaren sein, dass dies sehr aufwendig ist und viel Zeit zum Lernen benötigt.

Meine Ergebnisse waren mal mehr, mal weniger zufriedenstellend. Vor allem PER und Noisy haben mich etwas enttäuscht.

Etwas, das ich noch nicht gemacht habe, das aber sicher auch einen grossen Effekt hätte, ist das Hyperparameter Tuning. Ich habe das ausgelassen, um möglichst viele andere Erweiterungen zu sehen. Am Ende habe ich aber gemerkt, dass viele Experimente wahrscheinlich noch etwas besser gewesen wären, wenn ich die Parameter am Anfang etwas optimiert und mir die Veränderungen angesehen hätte. Gerade beim Frameskipping bin ich mir beispielsweise unsicher, ob sich dieser Trade-Off gelohnt hat.

Mit der Auswertung bin ich mässig zufrieden. Ich bin mir nicht ganz sicher, wie gut und aussagekräftig eine Auswertung mit einem kleinen Epsilon ist. Aber ich hatte das Problem, dass ich immer die gleichen Werte bekam, wenn Epsilon gleich Null war.

Auch das Logging im Tensoboard könnte noch erweitert werden. Zum Beispiel wie sich die Q-Werte oder die Schrittlänge während des Trainings verändern.

Diese Verbesserungen könnten für ein zukünftiges Projekt noch umgesetzt werden. Für dieses Projekt ist dies aufgrund der langen Laufzeit des DQN leider nicht mehr realisierbar.

Ausblick

In einem weiteren Schritt müssten sicherlich noch die Hyperparameter optimiert werden. Ich könnte mir auch gut vorstellen, dass die verschiedenen Erweiterungen gemischt werden. Zum Beispiel könnte man am Anfang mit einem Noisy Network arbeiten oder zeitweise mit einem PER Buffer und das Ganze mit einem Dueling Network verbinden.

Auch in der Auswertung könnten mehrere gleiche Agenten trainiert werden, um z.B. eine Aussage über die Unterschiede im Training machen zu können.